

Sydney Housing Price Prediction and Decision Support System

SIT307 Machine Learning: ML Mini Project (Distinction task) · Raghav · October 2026

Code and data: [\[insert GitHub / OneDrive link to the project ZIP here\]](#)

1. Problem definition and data collection

A real estate agency needs quick, evidence-based price estimates for appraisals and for checking offers: a supervised regression problem with **sale price** as the target. Because the markets differ so much, I model **log(price)** and judge models mainly on **MAPE**, so a \$100k miss on a \$500k unit counts more than on a \$5m house.

I chose three suburbs where I would consider buying, each a distinct market: **Bondi** (beachside, 7 km from the CBD, very high land values), **Parramatta** (Sydney's second CBD, high-rise apartments plus older houses on large blocks) and **Penrith** (outer west, the most affordable, newer apartment precincts and family houses). I expected location to dominate, then dwelling type (a house includes its land), then size.

I collected sales from Domain.com.au's public sold-listing pages, filtered to disclosed prices, recording price, date, sale method, bedrooms, bathrooms, parking, area, property type and listing URL. Spot checks against four individual listings matched exactly. Of 125 collected sales, I removed one duplicate listing and six retirement-village units (leasehold, not market prices), leaving **118 sales** (Bondi 38, Parramatta 38, Penrith 42; Sep 2025 – Sep 2026).

Quality and bias. Area was missing for 60% of records, and for apartments it sometimes recorded the whole building's site (2,364 m² for a 2-bed flat). Domain labelled some apartments as "House". About half of all sales withhold their price, so the data may under-represent disappointing and prestige results. I also pulled house-filtered pages to balance the mix, so suburb medians here are not market medians, and older records are mostly houses. Condition, views and floor level are not captured at all.

Suburb	Dwelling class	Sales	Median	Range
Bondi	house	17	\$4,050,000	\$2,750,000 – \$6,308,000
Bondi	unit	21	\$1,520,000	\$650,000 – \$2,600,000
Parramatta	house	18	\$1,582,500	\$1,150,000 – \$2,650,000
Parramatta	townhouse/villa	2	\$785,000	\$720,000 – \$850,000
Parramatta	unit	18	\$607,500	\$320,000 – \$855,000
Penrith	house	15	\$1,000,000	\$802,000 – \$1,500,000
Penrith	townhouse/villa	3	\$735,000	\$720,000 – \$820,000
Penrith	unit	24	\$558,500	\$440,000 – \$740,000

Table 1. Cleaned dataset by suburb and dwelling class (houses include semis and duplexes; apartments labelled "House" are reclassified as units).

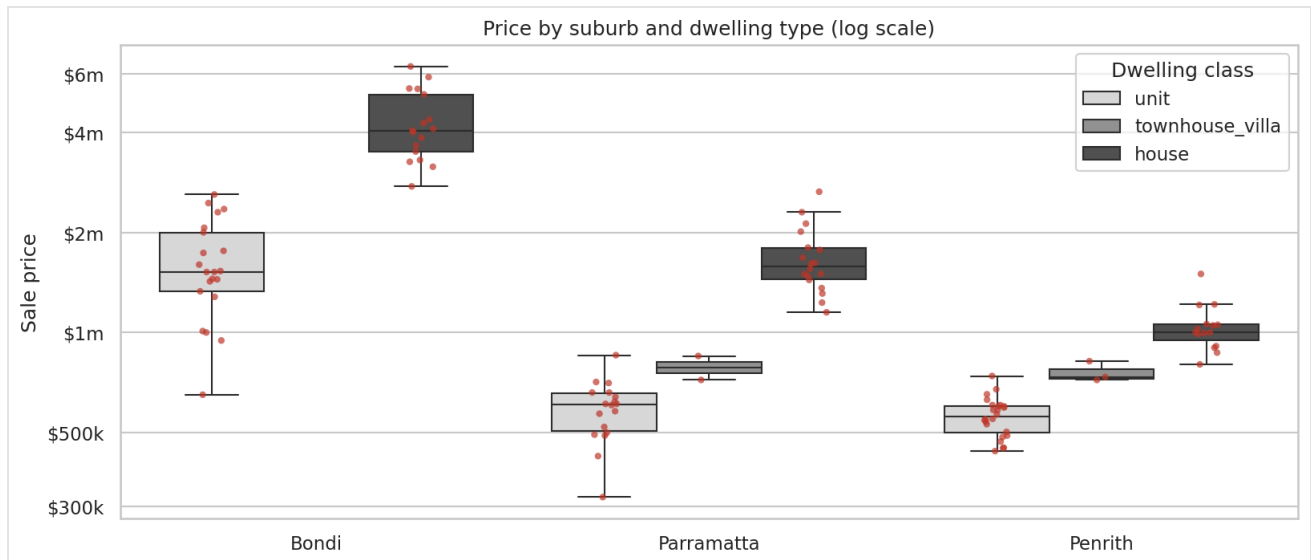


Figure 1. Price by suburb and dwelling type (log scale). The median house sells for about 2.6–2.7× the median unit in Bondi and Parramatta, but only 1.8× in Penrith.

2. Data understanding and feature engineering

Prices are strongly right-skewed (skewness 1.81, falling to 0.57 after a log transform) and form separate clusters by suburb and type. Within each suburb, price rises steadily with bedrooms (Spearman 0.82–0.89). After adjusting for suburb and type, prices were essentially flat over the window (−1.8% a year). Six robust-z outliers (e.g. a hotel apartment, a dual-occupancy block) were genuine sales, so I kept them.

Hypothesis, stated before engineering: the three strongest drivers are **suburb**, **dwelling type** and **bedrooms**. I engineered: a dwelling class that corrects Domain's mislabels; a **suburb** × **class** interaction, since the house premium differs by suburb; land area for land-holding dwellings only, plus a "known" flag; baths per bedroom; an auction flag; and months since the start of the window. I left out distance to the CBD because, with only three suburbs, it just relabels the suburb. With the same Ridge model, these features cut MAPE from 18.8% to **15.0%**, which matches my view that location and type, *and how they combine*, set price.

3. Model development and evaluation

I compared three approaches: **Ridge regression** (linear and stable, but cannot bend to every segment), **Random Forest** (bagging; captures interactions, averages away variance) and **Gradient Boosting** (boosting; often most accurate, but the easiest to overfit). Before training, I predicted the Random Forest would be slightly best, Ridge close behind, and Gradient Boosting the most overfit. I used **nested cross-validation**: 5-fold CV repeated 3 times, with hyperparameters tuned by an inner 5-fold grid search inside each outer fold, against a segment-median baseline.

Model	Train MAPE	Validation MAPE	Validation MAE	Validation RMSE	Validation R ²
Baseline: segment median	16.1%	17.5% ± 2.1	\$284,539	\$462,662	0.855
Ridge regression	12.9%	15.2% ± 2.1	\$245,504	\$388,217	0.891
Random Forest	6.9%	14.7% ± 2.7	\$234,012	\$372,790	0.903
Gradient Boosting	7.9%	14.6% ± 2.1	\$228,312	\$351,709	0.908

Table 2. Nested cross-validation, 15 outer folds (mean ± SD). Prices are back-transformed from log scale.

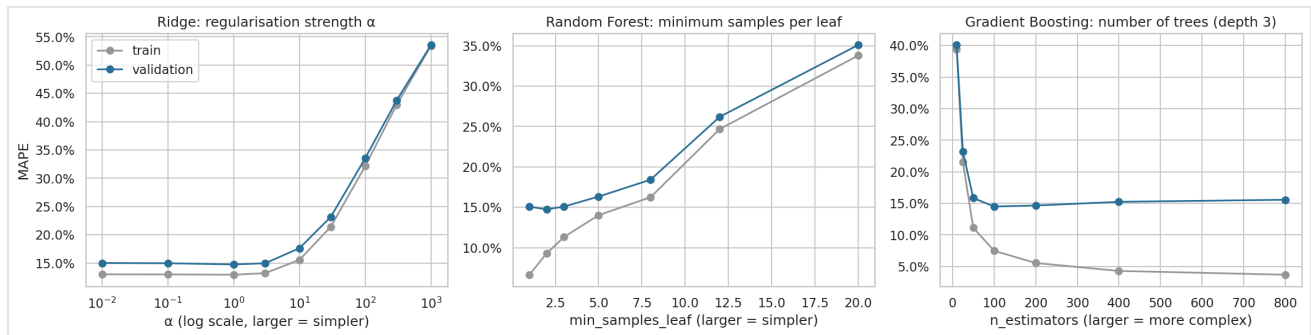


Figure 2. Validation curves: training vs validation MAPE as each model's complexity changes.

All models beat the baseline (Ridge in 14 of 15 folds), but the differences *between* models are within noise: Gradient Boosting beats Ridge in only 7 of 15 paired folds. **Ridge** slightly underfits, with a small train–validation gap. Both tree models nearly memorise the training folds: the **Random Forest** has the largest gap (6.9% vs 14.7%), and **Gradient Boosting** shows the classic curve, underfitting at 10–25 trees, best near 100, then overfitting as training error falls to 3.7%. Permutation importance confirmed suburb and dwelling type as the top drivers. Bedrooms mattered, but shared their size signal with bathrooms. Land area added little because it was mostly missing. My predictions were therefore partly right: Ridge was competitive, but Gradient Boosting overfit less than the forest.

Recommendation: Gradient Boosting. It is statistically tied on MAPE, but it has the lowest RMSE (about 9% below Ridge), so it makes fewer large dollar misses on high-value homes, and it is clearly better on Parramatta houses (13.2% vs 17.9%). The cost is interpretability, and the fact that it cannot extrapolate beyond the prices it has seen.

4. Investigating prediction failures

I read each of the five largest out-of-fold errors against its listing page. Four are under-predictions at the top of the market, and each is driven by information the model never sees:

#	Property	Features	Actual	Predicted	Error	Explanation from the listing
1	344 Birrell St, Bondi	House 5/3/1	\$5.86m	\$3.42m	−42%	Newly completed home; only three other 5+ bed Bondi houses in the data
2	62 Watson St, Bondi	House 3/2/1	\$3.30m	\$4.92m	+49%	Actually a semi-detached character home, treated as a freestanding house
3	5601/34 Wellington St, Bondi	Unit 2/2/1	\$2.60m	\$1.61m	−38%	Top floor, north-facing, harbour and beach views, new luxury building
4	124 Thomas St, Parramatta	House 3/1/1, 739 m ²	\$2.30m	\$1.36m	−41%	Riverside development site; value is land and zoning, not the house
5	11 Stephen St, Bondi	House 3/1/1	\$4.10m	\$3.21m	−22%	Original freestanding cottage; one bathroom signals renovation upside, not low value

Table 3. Five largest absolute out-of-fold errors (Gradient Boosting). Features are beds/baths/parking.

Ranked by percentage, the worst cases are cheap and unusual: the hotel apartment (+50%) and a 1-bed walk-up with no parking (+52%). Overall, 76% of sales were predicted within $\pm 20\%$. The missing information is **floor area, condition and age, views and floor level, zoning, attached vs freestanding**, and the agent's description, which explained every failure above. Predictions should be treated cautiously for prestige, new-build, development, serviced and rare property types, and outside these suburbs and dates. Homogeneous stock (Penrith apartments, 7–8% error) is inherently easier than heterogeneous Bondi stock (15–20%).

5. Deployment and reflection

The trained pipeline is saved with `joblib`, with metadata: cross-validated error, the 10th–90th percentile of out-of-fold errors (for a likely range) and the training ranges. I built the app in **Flask**, which the brief allows: Streamlit could not be installed in my build environment, and Flask let me run, test and screenshot the app. It imports the same `housing_features.py` as the notebook, so features are computed identically. It offers a single-property form, CSV upload with downloadable results, and a JSON API. Every estimate comes with a likely range, the three most similar real sales, and caution flags (Figures 3–4; instructions in Appendix A).

Reflection. Data work changed the results more than model choice (about 4 MAPE points from features, under 1 between models), and the baseline showed ML adds roughly 3 points. On small data, nested, repeated CV was essential: a single split could have crowned any model. In deployment, consistent features and honest uncertainty mattered as much as accuracy, and Ridge would be easier to explain. On bias and fairness: the sample over-represents disclosed sales; the model under-values premium homes and over-values cheap atypical units, which would compress real price differences if used for pricing or lending; and suburb, the strongest feature, correlates with income and demographics, so the model can reinforce existing geographic price gaps. It should support, not replace, a valuer. With more resources, I would use the NSW Valuer General's bulk sales data (including withheld prices), more suburbs with geocoded distances, text features from agent descriptions, floor area and zoning, conformal prediction intervals, and a wider tuning search.

Sydney Housing Price Estimator
Bondi · Parramatta · Penrith | Gradient Boosting trained on 118 sales (2025-09 to 2026-09) | typical error ±15%

Single property Upload CSV

Property details

Suburb
Parramatta

Property type
House

Bedrooms: 4 Bathrooms: 2 Parking: 2

Land size (m²), optional
750

Used for houses, townhouses and villas only.

Sale method
auction

Expected sale month
2026-10

Estimate price

Estimated sale price · house in Parramatta
\$1,665,000
Likely range **\$1,313,000 to \$2,049,000**

10th percentile 90th percentile
▲ estimate
In cross-validation, 8 in 10 properties sold inside a range of this width around the model's estimate.

⚠ Large blocks may carry development value the model cannot see (it under-valued a 739 m² Parramatta development site by 41%).

Most similar sales in the data

57 Pitt Street, Parramatta 4 bed · 2 bath · House · sold May 2026	\$1,310,000
13 Morton Street, Parramatta 4 bed · 3 bath · House · sold Apr 2026	\$1,800,000
58 Crimea Street, Parramatta 3 bed · 2 bath · House · sold Feb 2026	\$1,620,000

This is a decision-support estimate from 118 Domain.com.au sales, not a valuation. It cannot see condition, renovations, views, floor level or development potential, which is where its largest errors occurred.

SIT307 ML mini project · Gradient Boosting on log(price) · features: suburb, dwelling class, bedrooms, bathrooms, parking, land size, sale method, sale month.

Figure 3. Single-property estimate: a 4-bed Parramatta house on 750 m². The app shows the likely range, comparable sales and a large-block caution flag.

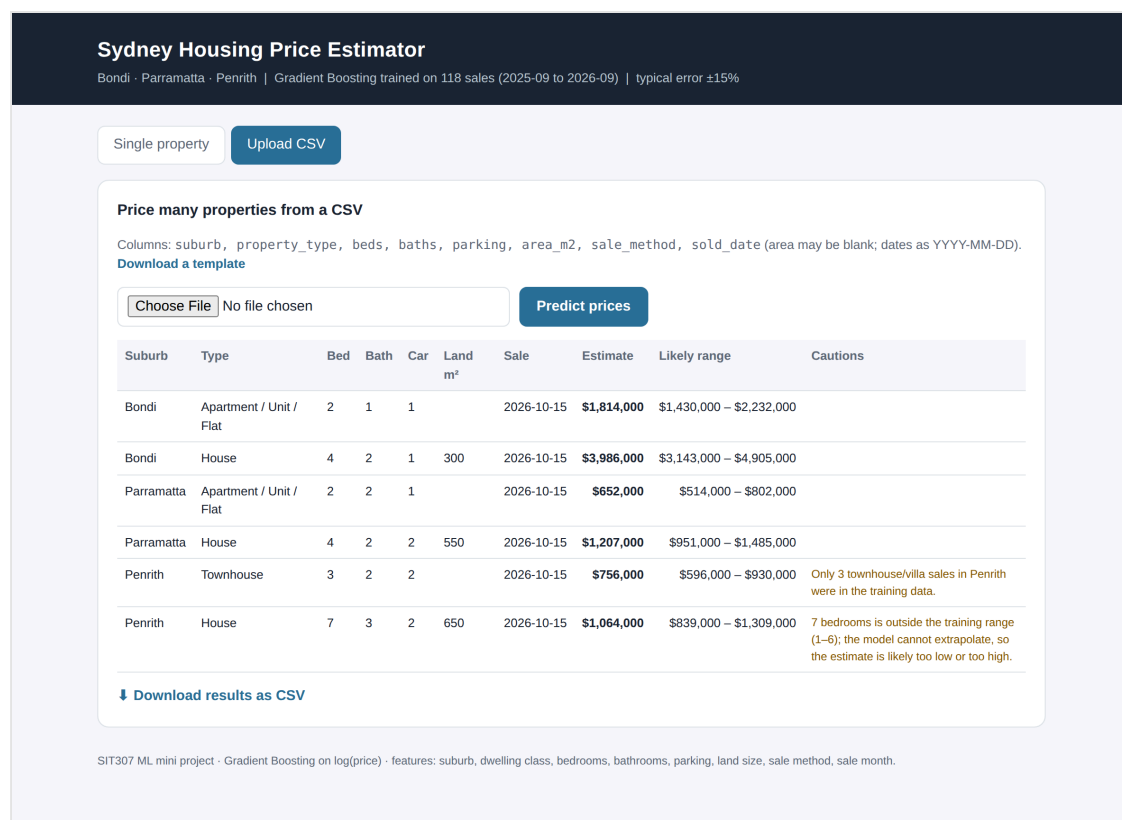


Figure 4. CSV upload: six properties priced at once, with caution flags for a thin segment and a 7-bedroom house outside the training range.

GenAI acknowledgement. Claude (Anthropic) was used as an assistant to help read Domain sold-listing pages and transcribe them into the dataset (records were spot-checked against the original listings), to draft and debug the Python code for the notebook and Flask app, and to draft explanatory text. All design choices, results and interpretations were reviewed by me, and I am responsible for the final submission.

References

- Domain.com.au (2026) *Sold listings: Bondi NSW 2026, Parramatta NSW 2150, Penrith NSW 2750*, accessed 30 Sep – 1 Oct 2026. Individual listing URLs are in `data/sydney_sold_raw.csv`.
- Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32.
- Friedman, J.H. (2001) 'Greedy function approximation: a gradient boosting machine', *Annals of Statistics*, 29(5), pp. 1189–1232.
- Hoerl, A.E. and Kennard, R.W. (1970) 'Ridge regression: biased estimation for nonorthogonal problems', *Technometrics*, 12(1), pp. 55–67.
- Varma, S. and Simon, R. (2006) 'Bias in error estimation when using cross-validation for model selection', *BMC Bioinformatics*, 7, 91.
- NSW Valuer General (2026) *Bulk property sales information*, <https://www.valuergeneral.nsw.gov.au> (suggested data source for future work).

Appendix A. How to build, run and use the code and app

Setup (Python 3.10+): unzip the project (or clone the repository), then in the project folder run `python -m venv .venv`, activate it, and run `pip install -r requirements.txt`.

Dataset: `python data/build_dataset.py` merges the raw collection batches into `data/sydney_sold_raw.csv` (already included).

Notebook: open `housing_price_prediction.ipynb` and choose *Kernel* → *Restart & Run All*, or run `jupyter nbconvert --to notebook --execute --inplace housing_price_prediction.ipynb`. It takes 3–5 minutes and writes the cleaned data, figures and the model to `app/model/`. All randomness is seeded (42).

Web app: run `python app/app.py` and open `http://127.0.0.1:5000`.

- *Single property:* choose suburb and property type; enter bedrooms, bathrooms, parking, optional land size (houses, townhouses and villas only), sale method and expected sale month; press **Estimate price**. Read the estimate together with its likely range, the comparable sales and any caution flags.
- *Upload CSV:* click the **Upload CSV** tab, download the template (or use `app/example_properties.csv`), fill in one property per row (suburb, property_type, beds, baths, parking, area_m2, sale_method, sold_date), upload it and press **Predict prices**; then use **Download results as CSV**.
- *API:* POST `/api/predict` with a JSON object (or list) using the same fields.

Figure A1. App start page.

Figure A2. Form filled in before pressing Estimate price.

Appendix B. Key code

B1. Feature engineering shared by notebook and app (housing_features.py)

```
"""
Shared cleaning and feature-engineering code for the Sydney housing project.

Both the notebook (training) and the web app (prediction) import this file,
so a property is turned into model inputs in exactly the same way in both.
"""
from __future__ import annotations

import numpy as np
import pandas as pd

SUBURBS = ["Bondi", "Parramatta", "Penrith"]
DWELLING_CLASSES = ["house", "townhouse_villa", "unit"]

# Domain's raw "property type" labels -> three broad dwelling classes.
TYPE_TO_CLASS = {
    "House": "house",
    "Semi-detached": "house",
    "Duplex": "house",
    "Townhouse": "townhouse_villa",
    "Villa": "townhouse_villa",
    "Apartment / Unit / Flat": "unit",
}

# Reference date for the time-trend feature (first month in the dataset).
START_MONTH = pd.Timestamp("2025-09-01")

NUMERIC_FEATURES = ["beds", "baths", "parking", "land_area_m2",
                    "baths_per_bed", "months_since_start"]
BINARY_FEATURES = ["is_auction", "land_area_known"]
CATEGORICAL_FEATURES = ["suburb", "dwelling_class"]
MODEL_FEATURES = NUMERIC_FEATURES + BINARY_FEATURES + CATEGORICAL_FEATURES

def dwelling_class(property_type: str, address: str) -> str:
    """Map a Domain property type to a dwelling class.

    Domain sometimes labels strata apartments as 'House' (e.g. a top-floor
    apartment at 9/27 Castlefield St). A 'House' whose address carries a
    unit number ("9/27 ...") is therefore treated as a unit.
    """
    cls = TYPE_TO_CLASS.get(property_type, "unit")
    if cls == "house" and "/" in str(address):
        cls = "unit"
    return cls

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    """Turn cleaned listing rows into model-ready features.

    Expects columns: suburb, property_type, address, beds, baths, parking,
    area_m2, sale_method, sold_date. Works for one row (the app) or many.
    """
    out = pd.DataFrame(index=df.index)
    out["suburb"] = df["suburb"].astype(str)
    out["dwelling_class"] = [dwelling_class(t, a)
                            for t, a in zip(df["property_type"], df["address"])]

    out["beds"] = pd.to_numeric(df["beds"], errors="coerce")
    out["baths"] = pd.to_numeric(df["baths"], errors="coerce")
    out["parking"] = pd.to_numeric(df["parking"], errors="coerce")
    out["baths_per_bed"] = out["baths"] / out["beds"].clip(lower=1)

    # Land area only means something for land-holding dwellings. Unit "areas"
    # on Domain are a mix of internal floor area and the whole strata site
    # (we saw 1,751 m2 and 2,364 m2 for 2-bed flats), so they are dropped.
    area = pd.to_numeric(df["area_m2"], errors="coerce")
    is_land = out["dwelling_class"].isin(["house", "townhouse_villa"])
    out["land_area_m2"] = area.where(is_land)
```

```

out["land_area_known"] = out["land_area_m2"].notna().astype(int)

out["is_auction"] = df["sale_method"].astype(str).str.contains(
    "auction", case=False).astype(int)
sold = pd.to_datetime(df["sold_date"])
out["months_since_start"] = ((sold.dt.year - START_MONTH.year) * 12
                             + (sold.dt.month - START_MONTH.month)).astype(float)

return out[MODEL_FEATURES]

def add_segment(X: pd.DataFrame) -> pd.DataFrame:
    """Add the suburb x dwelling-class interaction ("Bondi_house", ...).

    Used as the first step of every model pipeline rather than as a stored
    column, so permuting `suburb` or `dwelling_class` (permutation importance)
    also changes the interaction, and the app only has to supply raw inputs.
    """
    X = X.copy()
    X["segment"] = X["suburb"].astype(str) + "_" + X["dwelling_class"].astype(str)
    return X

```

B2. Model pipelines and nested cross-validation (from the notebook)

```

def preprocessor(scale_numeric):
    num_steps = [("impute", SimpleImputer(strategy="median"))]
    if scale_numeric:
        num_steps.append(("scale", StandardScaler()))
    return Pipeline([
        ("segment", FunctionTransformer(hf.add_segment)),
        ("columns", ColumnTransformer([
            ("num", Pipeline(num_steps), hf.NUMERIC_FEATURES),
            ("bin", "passthrough", hf.BINARY_FEATURES),
            ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
             hf.CATEGORICAL_FEATURES + ["segment"]),
        ], verbose_feature_names_out=False)),
    ])

def log_target(pipe):
    # Fit on log(price), predict back in dollars, so every metric is in $ terms
    return TransformedTargetRegressor(regressor=pipe, func=np.log, inverse_func=np.exp)

# --- the three models (tuned by an inner 5-fold GridSearchCV inside each outer fold) ---
inner = KFold(n_splits=5, shuffle=True, random_state=SEED)
models = {
    "Ridge": GridSearchCV(
        log_target(Pipeline([("prep", preprocessor(True)), ("model", Ridge())])),
        {"regressor__model__alpha": [0.1, 0.3, 1, 3, 10, 30]},
        cv=inner, scoring="neg_mean_absolute_percentage_error"),
    "Random Forest": GridSearchCV(
        log_target(Pipeline([("prep", preprocessor(False)),
                             ("model", RandomForestRegressor(n_estimators=300, random_state=SEED))])),
        {"regressor__model__min_samples_leaf": [1, 3, 5],
         "regressor__model__max_features": [0.33, 0.66, 1.0]},
        cv=inner, scoring="neg_mean_absolute_percentage_error", n_jobs=-1),
    "Gradient Boosting": GridSearchCV(
        log_target(Pipeline([("prep", preprocessor(False)),
                             ("model", GradientBoostingRegressor(learning_rate=0.05, subsample=0.8,
                                                                     random_state=SEED))])),
        {"regressor__model__n_estimators": [100, 300],
         "regressor__model__max_depth": [2, 3]},
        cv=inner, scoring="neg_mean_absolute_percentage_error", n_jobs=-1),
}

outer = RepeatedKFold(n_splits=5, n_repeats=3, random_state=SEED)

```

B3. Prediction in the app (app/app.py)

```

def predict_frame(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    if "address" not in df.columns:
        df["address"] = ""
    df["address"] = df["address"].fillna("")
    feats = hf.engineer_features(df)

```

```
pred = MODEL.predict(feats)
out = df[CSV_COLUMNS].copy()
out["predicted_price"] = pred.round(-3)
out["likely_low"] = (pred * np.exp(Q_LOW)).round(-3)
out["likely_high"] = (pred * np.exp(Q_HIGH)).round(-3)
out["cautions"] = [" | ".join(caution_flags(r, f)) for (_, r), (_, f) in zip(df.iterrows(),
feats.iterrows())]
return out, feats
```

The complete code, all outputs and every figure are in the executed notebook `housing_price_prediction.ipynb` and the project repository.